

Error bars for the pCN sampler: batch means (blocking) and the autocorrelation time

1 The problem

The pCN sampler produces, per MPI core, a Markov chain of mean-field trajectories V and from it a stream of single-sample observables

$$o_1, o_2, \dots, o_L, \quad o_i = \frac{\text{Tr}[U(\beta - \tau; V_i) S^a U(\tau; V_i) S^b]}{Z(V_i)},$$

i.e. one such stream for every correlation point (a, b, τ_k) (real and imaginary part). The reported correlation is the global mean over all $N = C L$ samples (C = number of cores, each an independent chain),

$$\bar{o} = \frac{1}{N} \sum_{i=1}^N o_i.$$

We need the standard error of $\bar{o}$.

If the samples were independent, the squared standard error would be

$$\sigma_{\text{iid}}^2 = \frac{s^2}{N}, \quad s^2 = \frac{1}{N} \sum_i o_i^2 - \bar{o}^2 \quad (1)$$

with s^2 the marginal sample variance. But a Markov chain produces *autocorrelated* samples. Writing ρ_k for the normalised autocorrelation of o at lag k and

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{k \geq 1} \rho_k \quad (2)$$

for the integrated autocorrelation time (in units of samples; $\tau_{\text{int}} = \frac{1}{2}$ for independent draws), the true variance of the mean is inflated:

$$\text{Var}(\bar{o}) = \frac{s^2}{N} (2\tau_{\text{int}}) = \sigma_{\text{iid}}^2 \cdot 2\tau_{\text{int}}. \quad (3)$$

Equivalently the effective sample size is $N_{\text{eff}} = N/(2\tau_{\text{int}})$. Using (1) instead of (3) underestimates the error by $\sqrt{2\tau_{\text{int}}}$, which at low temperature is large (here $\tau_{\text{int}} \sim 10^3$, Sec. 6). The previous estimator scaled σ_{iid} by an AR(1) factor inferred from the acceptance rate; that captures only the lag-1 term ρ_1 and badly underestimates τ_{int} when the chain has a slow mode. The fix is to measure $\text{Var}(\bar{o})$ directly, without modelling ρ_k .

2 Batch means (blocking)

Split each chain’s L samples into B consecutive blocks of length $\ell = \lfloor L/B \rfloor$ and form the block means

$$\bar{o}_b = \frac{1}{\ell} \sum_{i \in \text{block } b} o_i, \quad b = 1, \dots, B. \quad (4)$$

Adjacent samples within a block are correlated, but two block means become *independent* once the block length exceeds the autocorrelation time, $\ell \gg \tau_{\text{int}}$. Pooling the block means of all cores gives $M = CB$ approximately independent estimates of the same quantity, and the standard error of their average — which is exactly the global mean $\bar{o}$, since the blocks tile the data with equal weight — is

$$\hat{\sigma}_{\text{block}}^2 = \frac{1}{M} \widehat{\text{Var}}(\{\bar{o}_m\}_{m=1}^M) = \frac{1}{M} \cdot \frac{1}{M-1} \sum_{m=1}^M (\bar{o}_m - \bar{o})^2. \quad (5)$$

This is the batch-means estimator. It needs no model of the autocorrelation: the block length absorbs it. Crucially the statistics come from the *number of blocks* $M = CB$, not from the number of cores C , so it is usable with few cores (here 4–16 cores $\times$ 32 blocks = 128–512 pooled block means).

Pooling. Each core flattens its B block means; an `MPI_Allgather` collects all CB of them on every rank, which then evaluate (5) per correlation point.

3 Validity check: the blocking curve and the plateau

Equation (5) is only correct when $\ell \gg \tau_{\text{int}}$; for too-short blocks it *underestimates* the error. The Flyvbjerg–Petersen test makes this visible: recompute $\hat{\sigma}_{\text{block}}$ for a sequence of block lengths by repeatedly merging adjacent blocks *within each chain*, $\ell \rightarrow 2\ell \rightarrow 4\ell \rightarrow \dots$ (which keeps the merged blocks contiguous in chain time). The estimate rises with ℓ and then *plateaus* once $\ell \gg \tau_{\text{int}}$:

$$\ell \ll \tau_{\text{int}} : \hat{\sigma}_{\text{block}} \text{ too small} \quad \longrightarrow \quad \ell \gg \tau_{\text{int}} : \hat{\sigma}_{\text{block}} \rightarrow \text{true } \sigma(\bar{o}).$$

The code therefore does *not* report the finest binning. For each point it takes the **maximum** of $\hat{\sigma}_{\text{block}}$ over all merge levels whose pooled block count stays ≥ 8 (the level $\ell = L/B$ is always included). The curve is monotone-then-noisy, so the maximum is a robust read of the plateau, and it never under-covers. The aggregate curve (mean over the computed `Reg` points vs. block length) is written to the output so the plateau can be inspected; the printed “blocking plateau ratio” (longest/shortest level) is ≈ 1 when the blocks are long enough and $\gg 1$ when the run is too short for its autocorrelation.

4 Recovering the autocorrelation time

The blocking error and the i.i.d. error differ precisely by the inflation factor (3). Once the blocking estimate has plateaued it equals the true $\text{Var}(\bar{o})$, so the ratio gives τ_{int} directly:

$$2\tau_{\text{int}} = \frac{\text{Var}(\bar{o})}{\sigma_{\text{iid}}^2} = \frac{\hat{\sigma}_{\text{block}}^2}{s^2/N} \quad \Longrightarrow \quad \tau_{\text{int}} = \frac{1}{2} \frac{\hat{\sigma}_{\text{block}}^2 N}{s^2}. \quad (6)$$

Both ingredients are already in hand: $\hat{\sigma}_{\text{block}}$ from (5) and the marginal variance s^2 from the running $\sum_i o_i^2$ (the `sample_sqsum` accumulator), so τ_{int} costs nothing extra. It is computed per correlation point.

Interpretation. τ_{int} from (6) is in units of pCN steps (samples) and is a property of a *single* chain: because the cores are independent, $N_{\text{eff}} = C L / (2\tau_{\text{int}}) = N / (2\tau_{\text{int}})$, and the C -dependence cancels in (6). The independent limit gives $\tau_{\text{int}} = \frac{1}{2}$; a large value means consecutive samples stay correlated over $\sim \tau_{\text{int}}$ steps. This is the number that sets the practical rules of thumb:

- **burn-in** $\gtrsim 5\text{--}10 \tau_{\text{int}}$ (forget the initial condition);
- **per-core chain length** $L \gtrsim 10\text{--}20 \tau_{\text{int}}$ (so each chain is well mixed and unbiased, and (5) has plateaued);
- under slow mixing prefer *long* chains over *many short* ones: initialization bias does not average across cores, and burn-in is paid per chain.

5 In the code

```
--errmethod  blocking (default, this note) or ar1 (legacy acceptance factor)
--numBlocks  number of blocks per core  $B$  (default 32)
```

Per-sample accumulation into the current block and block closing live in `Run_Time_Data (init_blocks, close_block)` fed by `compute_spin_correlations`; the pooling, plateau and τ_{int} are in `compute_sample_stds_block`. Written to the HDF5 `runtimedata` group:

- `Re/Im_correlation_sample_stds` — the plateau blocking error (5);
- `Re/Im_correlation_tau_int` — τ_{int} in steps (6);
- `blocking_curve_block_length, blocking_curve_sigma` — the Flyvbjerg–Petersen curve;
- `attribute error_method`.

Each iteration prints the “blocking plateau ratio” and “max tau_int (steps)”.

6 Measured behaviour ($\beta = 12$, ISO, symmetry type C)

The slow mode shows up in the should-be-zero $\text{Re } g^{xy}(\tau)$. Validated against the spread of independent runs (the ground-truth error):

- $\tau_{\text{int}} \approx 2800$ steps. This explains the sample-size dependence: at $L = 10^4/\text{core}$ the block length $L/B \approx 310$ is $\ll \tau_{\text{int}}$ and the error is underestimated; at $L = 10^5/\text{core}$, $L/B \approx 3100 \gtrsim \tau_{\text{int}}$ and the estimator is calibrated.
- At $L = 10^5/\text{core}$ the reported-to-true error ratio is ≈ 1.0 , and $\text{Re } g^{xy}$ is statistically consistent with zero ($\lesssim 2\sigma$). The apparent “symmetry violation” seen with the AR(1) error was an underestimated error bar, not broken physics.
- The between-run spread falls $\sim 1/\sqrt{N}$ as samples grow, i.e. the chain is *slow but ergodic*, not trapped behind a barrier — more samples (and burn-in $\gtrsim 5\text{--}10 \tau_{\text{int}}$) suffice; tempering is not required.